JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class StreamTokenizer

java.lang.Object
    java.io.StreamTokenizer

```
public class StreamTokenizer
extends Object
```

The `StreamTokenizer` class takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time. The parsing process is controlled by a table and a number of flags that can be set to various states. The stream tokenizer can recognize identifiers, numbers, quoted strings, and various comment styles.

Each byte read from the input stream is regarded as a character in the range `'\u0000'` through `'\u00FF'`. The character value is used to look up five possible attributes of the character: *white space*, *alphabetic*, *numeric*, *string quote*, and *comment character*. Each character can have zero or more of these attributes.

In addition, an instance has four flags. These flags indicate:

- Whether line terminators are to be returned as tokens or treated as white space that merely separates tokens.
- Whether C-style comments are to be recognized and skipped.
- Whether C++-style comments are to be recognized and skipped.
- Whether the characters of identifiers are converted to lowercase.

A typical application first constructs an instance of this class, sets up the syntax tables, and then repeatedly loops calling the `nextToken` method in each iteration of the loop until it returns the value `TT_EOF`.

Since:

1.0

See Also:

`nextToken()`, `TT_EOF`

Field Summary

Fields

Modifier and Type	Field	Description
`double`	**`nval`**	If the current token is a number, this field contains the value of that number.

String	**sval**	If the current token is a word token, this field contains a string giving the characters of the word token.
static final int	**TT_EOF**	A constant indicating that the end of the stream has been read.
static final int	**TT_EOL**	A constant indicating that the end of the line has been read.
static final int	**TT_NUMBER**	A constant indicating that a number token has been read.
static final int	**TT_WORD**	A constant indicating that a word token has been read.
int	**ttype**	After a call to the nextToken method, this field contains the type of the token just read.

Constructor Summary

Constructors

Constructor	Description

StreamTokenizer(InputStream is)	**Deprecated.**
	As of JDK version 1.1, the preferred way to tokenize an input stream is to convert it into a character stream, for example:
	`Reader r = new BufferedReader(new InputStrea`
	`StreamTokenizer st = new StreamTokenizer(r);`

StreamTokenizer(Reader r)	Create a tokenizer that parses the given character stream.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	`commentChar(int ch)`	Specifies that the character argument starts a single-line comment.
void	`eolIsSignificant (boolean flag)`	Determines whether or not ends of line are treated as tokens.
int	`lineno()`	Returns the current line number.
void	`lowerCaseMode (boolean fl)`	Determines whether or not word token are automatically lowercased.
int	`nextToken()`	Parses the next token from the input stream of this tokenizer.
void	`ordinaryChar(int ch)`	Specifies that the character argument is "ordinary" in this tokenizer.
void	`ordinaryChars(int low, int hi)`	Specifies that all characters *c* in the range `low <= c <= high` are "ordinary" in this tokenizer.
void	`parseNumbers()`	Specifies that numbers should be parsed by this tokenizer.
void	`pushBack()`	Causes the next call to the `nextToken` method of this tokenizer to return the current value in the `ttype` field, and not to

		modify the value in the `nval` or `sval` field.
`void`	**`quoteChar`**`(int ch)`	Specifies that matching pairs of this character delimit string constants in this tokenizer.
`void`	**`resetSyntax`**`()`	Resets this tokenizer's syntax table so that all characters are "ordinary."
`void`	**`slashSlashComments`**`(boolean flag)`	Determines whether or not the tokenizer recognizes C++-style comments.
`void`	**`slashStarComments`**`(boolean flag)`	Determines whether or not the tokenizer recognizes C-style comments.
`String`	**`toString`**`()`	Returns the string representation of the current stream token and the line number it occurs on.
`void`	**`whitespaceChars`**`(int low, int hi)`	Specifies that all characters *c* in the range `low <= c <= high` are white space characters.
`void`	**`wordChars`**`(int low, int hi)`	Specifies that all characters *c* in the range `low <= c <= high` are word constituents.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

ttype

`public int ttype`

After a call to the `nextToken` method, this field contains the type of the token just read. For a single character token, its value is the single character, converted to an integer. For a quoted string token, its value is the quote character. Otherwise, its value is one of the following:

- `TT_WORD` indicates that the token is a word.
- `TT_NUMBER` indicates that the token is a number.
- `TT_EOL` indicates that the end of line has been read. The field can only have this value if the `eolIsSignificant` method has been called with the argument `true`.
- `TT_EOF` indicates that the end of the input stream has been reached.

The initial value of this field is -4.

See Also:

`eolIsSignificant(boolean)`, `nextToken()`, `quoteChar(int)`, `TT_EOF`, `TT_EOL`, `TT_NUMBER`, `TT_WORD`

TT_EOF

`public static final int TT_EOF`

A constant indicating that the end of the stream has been read.

See Also:

Constant Field Values

TT_EOL

`public static final int TT_EOL`

A constant indicating that the end of the line has been read.

See Also:

Constant Field Values

TT_NUMBER

```
public static final int TT_NUMBER
```

A constant indicating that a number token has been read.

See Also:

Constant Field Values

TT_WORD

```
public static final int TT_WORD
```

A constant indicating that a word token has been read.

See Also:

Constant Field Values

sval

```
public String sval
```

If the current token is a word token, this field contains a string giving the characters of the word token. When the current token is a quoted string token, this field contains the body of the string.

The current token is a word when the value of the `ttype` field is `TT_WORD`. The current token is a quoted string token when the value of the `ttype` field is a quote character.

The initial value of this field is null.

See Also:

`quoteChar(int)`, `TT_WORD`, `ttype`

nval

```
public double nval
```

If the current token is a number, this field contains the value of that number. The current token is a number when the value of the `ttype` field is `TT_NUMBER`.

The initial value of this field is 0.0.

See Also:

`TT_NUMBER`, `ttype`

Constructor Details

StreamTokenizer

```
@Deprecated
public StreamTokenizer(InputStream is)
```

Deprecated.
As of JDK version 1.1, the preferred way to tokenize an input stream is to convert it into a character stream, for example:

```
Reader r = new BufferedReader(new InputStreamReader(is));
StreamTokenizer st = new StreamTokenizer(r);
```

Creates a stream tokenizer that parses the specified input stream. The stream tokenizer is initialized to the following default state:

- All byte values `'A'` through `'Z'`, `'a'` through `'z'`, and `'\u00A0'` through `'\u00FF'` are considered to be alphabetic.
- All byte values `'\u0000'` through `'\u0020'` are considered to be white space.
- `'/'` is a comment character.
- Single quote `'\''` and double quote `'"'` are string quote characters.
- Numbers are parsed.
- Ends of lines are treated as white space, not as separate tokens.
- C-style and C++-style comments are not recognized.

Parameters:

`is` - an input stream.

See Also:

`BufferedReader`,
`InputStreamReader`,
`StreamTokenizer(java.io.Reader)`

StreamTokenizer

```
public StreamTokenizer(Reader r)
```

Create a tokenizer that parses the given character stream.

Parameters:

`r` - a Reader object providing the input stream.

Since:

1.1

Method Details

resetSyntax

```
public void resetSyntax()
```

Resets this tokenizer's syntax table so that all characters are "ordinary." See the `ordinaryChar` method for more information on a character being ordinary.

See Also:

`ordinaryChar(int)`

wordChars

```
public void wordChars(int low,
                      int hi)
```

Specifies that all characters *c* in the range `low <= c <= high` are word constituents. A word token consists of a word constituent followed by zero or more word constituents or number constituents.

Parameters:

`low` - the low end of the range.

`hi` - the high end of the range.

whitespaceChars

```
public void whitespaceChars(int low,
                            int hi)
```

Specifies that all characters *c* in the range `low <= c <= high` are white space characters. White space characters serve only to separate tokens in the input stream.

Any other attribute settings for the characters in the specified range are cleared.

Parameters:

`low` - the low end of the range.

`hi` - the high end of the range.

ordinaryChars

```
public void ordinaryChars(int low,
                          int hi)
```

Specifies that all characters *c* in the range `low <= c <= high` are "ordinary" in this tokenizer. See the `ordinaryChar` method for more information on a character being ordinary.

Parameters:

`low` - the low end of the range.

`hi` - the high end of the range.

See Also:

`ordinaryChar(int)`

ordinaryChar

```
public void ordinaryChar(int ch)
```

Specifies that the character argument is "ordinary" in this tokenizer. It removes any special significance the character has as a comment character, word component, string delimiter, white space, or number character. When such a character is encountered by the parser, the parser treats it as a single-character token and sets `ttype` field to the character value.

Making a line terminator character "ordinary" may interfere with the ability of a `StreamTokenizer` to count lines. The `lineno` method may no longer reflect the presence of such terminator characters in its line count.

Parameters:

`ch` - the character.

See Also:

`ttype`

commentChar

```
public void commentChar(int ch)
```

Specifies that the character argument starts a single-line comment. All characters from the comment character to the end of the line are ignored by this stream tokenizer.

Any other attribute settings for the specified character are cleared.

Parameters:

`ch` - the character.

quoteChar

```
public void quoteChar(int ch)
```

Specifies that matching pairs of this character delimit string constants in this tokenizer.

When the `nextToken` method encounters a string constant, the `ttype` field is set to the string delimiter and the `sval` field is set to the body of the string.

If a string quote character is encountered, then a string is recognized, consisting of all characters after (but not including) the string quote character, up to (but not including) the next occurrence of that same string quote character, or a line terminator, or end of file. The usual escape sequences such as `"\n"` and `"\t"` are recognized and converted to single characters as the string is parsed.

Any other attribute settings for the specified character are cleared.

Parameters:

`ch` - the character.

See Also:

`nextToken()`, `sval`, `ttype`

parseNumbers

```
public void parseNumbers()
```

Specifies that numbers should be parsed by this tokenizer. The syntax table of this tokenizer is modified so that each of the twelve characters:

```
    0 1 2 3 4 5 6 7 8 9 . -
```

has the "numeric" attribute.

When the parser encounters a word token that has the format of a double precision floating-point number, it treats the token as a number rather than a word, by setting the `ttype` field to the value `TT_NUMBER` and putting the numeric value of the token into the `nval` field.

See Also:

`nval`, `TT_NUMBER`, `ttype`

eolIsSignificant

```
public void eolIsSignificant(boolean flag)
```

Determines whether or not ends of line are treated as tokens. If the flag argument is true, this tokenizer treats end of lines as tokens; the `nextToken` method returns `TT_EOL` and also sets the `ttype` field to this value when an end of line is read.

A line is a sequence of characters ending with either a carriage-return character (`'\r'`) or a newline character (`'\n'`). In addition, a carriage-return character followed immediately by a newline character is treated as a single end-of-line token.

If the `flag` is false, end-of-line characters are treated as white space and serve only to separate tokens.

Parameters:

`flag` - `true` indicates that end-of-line characters are separate tokens; `false` indicates that end-of-line characters are white space.

See Also:

`nextToken()`, `ttype`, `TT_EOL`

slashStarComments

```
public void slashStarComments(boolean flag)
```

Determines whether or not the tokenizer recognizes C-style comments. If the flag argument is `true`, this stream tokenizer recognizes C-style comments. All text between successive occurrences of `/*` and `*/` are discarded.

If the flag argument is `false`, then C-style comments are not treated specially.

Parameters:

`flag` - `true` indicates to recognize and ignore C-style comments.

slashSlashComments

```
public void slashSlashComments(boolean flag)
```

Determines whether or not the tokenizer recognizes C++-style comments. If the flag argument is `true`, this stream tokenizer recognizes C++-style comments. Any occurrence of two consecutive slash characters (`'/'`) is treated as the beginning of a comment that extends to the end of the line.

If the flag argument is `false`, then C++-style comments are not treated specially.

Parameters:

`flag` - `true` indicates to recognize and ignore C++-style comments.

lowerCaseMode

```
public void lowerCaseMode(boolean fl)
```

Determines whether or not word token are automatically lowercased. If the flag argument is `true`, then the value in the `sval` field is lowercased whenever

a word token is returned (the `ttype` field has the value `TT_WORD`) by the `nextToken` method of this tokenizer.

If the flag argument is `false`, then the `sval` field is not modified.

Parameters:

`fl` - `true` indicates that all word tokens should be lowercased.

See Also:

`nextToken()`, `ttype`, `TT_WORD`

nextToken

```
public int nextToken()
             throws IOException
```

Parses the next token from the input stream of this tokenizer. The type of the next token is returned in the `ttype` field. Additional information about the token may be in the `nval` field or the `sval` field of this tokenizer.

Typical clients of this class first set up the syntax tables and then sit in a loop calling nextToken to parse successive tokens until TT_EOF is returned.

Returns:

the value of the `ttype` field.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`nval`, `sval`, `ttype`

pushBack

```
public void pushBack()
```

Causes the next call to the `nextToken` method of this tokenizer to return the current value in the `ttype` field, and not to modify the value in the `nval` or `sval` field.

See Also:

`nextToken()`, `nval`, `sval`, `ttype`

lineno

```
public int lineno()
```

Returns the current line number.

Returns:

the current line number of this stream tokenizer.

toString

```
public String toString()
```

Returns the string representation of the current stream token and the line number it occurs on.

The precise string returned is unspecified, although the following example can be considered typical:

```
        Token['a'], line 10
```

Overrides:

`toString` in class `Object`

Returns:

a string representation of the token

See Also:

`nval`, `sval`, `ttype`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.